

```

#include <iostream>
#include <signal.h>
#include <unistd.h>
#include <fcntl.h>
#include <sstream>
#include <map>
#include <termios.h>
#include <chrono>
#include <pigpio.h> // <<< added for ultrasonic

#include "sl_lidar.h"
#include "sl_lidar_driver.h"

using namespace sl;
ILidarDriver* lidar = nullptr;

// -----
// millis() Linux
// -----
unsigned long millis() {
    using namespace std::chrono;
    static steady_clock::time_point start = steady_clock::now();
    return (unsigned long)duration_cast<milliseconds>(steady_clock::now() - start).count();
}

// -----
// Serial config
// -----
void configSerialPort(int fd) {
    struct termios tty;
    tcgetattr(fd, &tty);
    cfmakeraw(&tty);
    cfsetispeed(&tty, B115200);
    cfsetospeed(&tty, B115200);
    tty.c_cflag &= ~CRTSCTS;
    tty.c_cflag |= CREAD | CLOCAL;
    tty.c_cflag &= ~PARENB;
    tty.c_cflag &= ~CSTOPB;
    tty.c_cflag &= ~CSIZE;
    tty.c_cflag |= CS8;
    tty.c_cc[VTIME] = 0;
    tty.c_cc[VMIN] = 0;
    tcflush(fd, TCIFLUSH);
    tcsetattr(fd, TCSANOW, &tty);
}

```

```

}

// -----
// Ctrl+C handler
// -----
void ctrlc_handler(int) {
    if (lidar) {
        lidar->stop();
        lidar->reset();
        delete lidar;
        lidar = nullptr;
    }
    gpioTerminate(); // cleanup pigpio
    exit(0);
}

// -----
// Single-pin ultrasonic read
// SIG on GPIO 22
// -----
float readUltrasonic(uint8_t pin) {
    // send 10µs pulse
    gpioSetMode(pin, PI_OUTPUT);
    gpioWrite(pin, 0);
    gpioDelay(2);    // 2µs
    gpioWrite(pin, 1);
    gpioDelay(10);   // 10µs pulse
    gpioWrite(pin, 0);

    // switch to input to read echo
    gpioSetMode(pin, PI_INPUT);

    // wait for rising edge
    uint32_t startTick = gpioTick();
    while (gpioRead(pin) == 0) {
        if ((gpioTick() - startTick) > 30000) return -1; // timeout ~30ms
    }
    uint32_t pulseStart = gpioTick();

    // wait for falling edge
    while (gpioRead(pin) == 1) {
        if ((gpioTick() - pulseStart) > 30000) return -1; // timeout
    }
    uint32_t pulseEnd = gpioTick();
}

```

```

float pulseWidth = float(pulseEnd - pulseStart); // in microseconds
float distance_cm = pulseWidth / 58.0f;          // convert to cm
return distance_cm;
}

// -----
// MAIN
// -----
int main() {
    signal(SIGINT, ctrlc_handler);

    // ---- pigpio init ----
    if (gpioInitialise() < 0) {
        std::cerr << "pigpio init failed!" << std::endl;
        return 1;
    }

    // ---- LIDAR init ----
    auto channel_res = createSerialPortChannel("/dev/ttyUSB0", 256000);
    IChannel* channel = *channel_res;

    auto driver_res = createLidarDriver();
    lidar = *driver_res;

    lidar->connect(channel);
    lidar->reset(5000);
    usleep(3000000);

    lidar->setMotorSpeed();
    lidar->startScan(false, false);
    lidar->setMotorSpeed(200);

    sl_lidar_response_measurement_node_hq_t nodes[8192];
    size_t count = sizeof(nodes)/sizeof(nodes[0]);

    while (true) {
        float sum_dist[360] = {0};
        int count_dist[360] = {0};

        count = sizeof(nodes)/sizeof(nodes[0]);
        lidar->grabScanDataHq(nodes, count, 0);
        lidar->ascendScanData(nodes, count);
    }
}

```

```

float avg0 = 0;
float maxValue = 0;
int maxAngle = 0;

for (size_t i = 0; i < count; ++i) {
    int angle = (nodes[i].angle_z_q14 * 90) >> 14;
    float dist = nodes[i].dist_mm_q2 / 4.0f;

    if ((angle >= 270 && angle < 360) || (angle >= 0 && angle <= 90)) {
        sum_dist[angle] += dist;
        count_dist[angle]++;
    }
}

for (int a = 0; a < 360; ++a) {
    if (count_dist[a] > 0) {
        float avg = sum_dist[a] / count_dist[a];
        if (a == 0) avg0 = avg;
        if (avg > maxValue) {
            maxValue = avg;
            maxAngle = a;
        }
    }
}

int sendAngle = (avg0 > 500) ? 0 : maxAngle;

// === MODE F/R : vitesse directe ===
int speed = 1500;

if (avg0 > 500) speed = 1680;
else if (avg0 >= 300) speed = 1650;
else if (avg0 >= 200) speed = 1630;
else if (avg0 >= 100) speed = 1620;
else {
    speed = 1350; // marche arriere directe
    sendAngle = (sendAngle + 180) % 360;
}

// ---- ULTRASONIC FLOOR SENSOR ----
float floorDist = readUltrasonic(22); // SIG = GPIO 22
if (floorDist > 0) {
    std::cout << "[ULTRA] floor object distance = " << floorDist << " cm" << std::endl;
}

```

```

    if (floorDist < 20.0) {
        std::cout << "[ULTRA] WARNING: object very close to floor!" << std::endl;
        // Optional: modify speed or stop robot here
        speed = 1200; // example slow down
    }
}

// === DEBUG ===
std::cout << "[LIDAR] dist_front=" << avg0
    << " | angle=" << sendAngle
    << " | speed=" << speed
    << (speed < 1500 ? " (REVERSE)" : " (FORWARD)")
    << std::endl;

// === JSON ===
std::stringstream ss;
ss << "{\"action\":\"drive\",\"angle\": " << sendAngle
    << ", \"speed\": " << speed << "}";

int serial_port = open("/dev/ttyUSB1", O_RDWR | O_NOCTTY | O_NDELAY);
if (serial_port >= 0) {
    fcntl(serial_port, F_SETFL, 0);
    configSerialPort(serial_port);
    std::string msg = ss.str() + "\n";
    write(serial_port, msg.c_str(), msg.size());
    close(serial_port);
}

usleep(50000);
}

gpioTerminate();
return 0;
}

```